ApacheCon 2003
WE14: Developing Web Applications with CGI::Application
Jason Purdy - jason@journalistic.com

Preface

This handout contains the core presentation notes and is provided for you to either go along with as I present, or use if you are unable to attend. I have provided some code examples in the Appendix, but there are some examples that I will develop as I present the course, so they are unable to be attached here. Please feel free to e-mail me at the address above for all of the code examples. If you are unable to attend, you will also miss out on asking questions that may help you further understand the material. Again, please feel free to e-mail with your questions and I will try to help.

Introduction

Before we dive into the material, let me disclaim that I don't have all the answers - I've been developing Web applications for a long time and I am excited to share my methodology with you, but like any wise developer, I also know that I still have a lot to learn and there are other (& possibly easier) ways to accomplish the same goal. I am honored and thankful for this opportunity to attend ApacheCon to learn from other distinguished speakers as well.

The Material

Now I don't want to get into much of a history lesson, but I thought it would be interesting to see how much the Web has evolved since the beginning. Perl was around when the Web took off (at version 4) and complemented the methodology that the Web architects came up with very nicely, with CGI (Common Gateway Interface). We would write simple scripts to read the standard input (STDIN), parse out the variables and put together an HTML response. I'm not sure if I want to even show you what that looks like, I'm so embarrassed.

Then a great effort was put together, headed by Lincoln Stein, to encapsulate all of the best practices of dealing with CGI requests into a perl module appropriately called CGI.pm. There was much rejoicing and it was adopted into the standard Perl library when you install Perl. Every once in a while, folks muse about rolling their own CGI parser, thinking they have a better way of doing things ... I believe those folks just have too much time on their hands.

Even with CGI.pm, though, scripts can become very unwieldy and hard to maintain as they grow in complexity, especially when you develop multiple-stage applications and encapsulate the HTML responses within your script. That's where CGI::Application and HTML::Template come into play.

CGI::Application is simply an object-oriented framework and provides support for CGI.pm (its parent class) methods as well as built-in support for HTML::Template. Other Perl developers have contributed to the CGI::Application framework by developing sibling or subsidiary classes that you can use in conjunction, such as CGI::Application::ValidateRM. I don't want to intimidate you this early on with all those possibilities, so just think of CGI::Application as a toolset for building your application. As you grow in your comfort, you can easily add-on new tools to reduce the amount of work you have to do and isn't that what good developers are all about?

After you trudge down the same path time & time again, you should look for shortcuts. This happened to me (though I won't admit after how many times I actually developed in “longhand”) and I picked up HTML::Template as my first step to enlightenment. It also helped me work better within my organization, as we have a graphics department that's responsible for our Web sites' look & feel. For some reason, my organization didn't trust me with the graphics aspect. They would develop the page in

static HTML and I would copy/paste that into my script. That was the before picture ... after HTML::Template, I would develop a spartan template page - enough to get my prototype to work and then turn the template page over to them so they can work their magic. Two quick tips: make a backup copy of the template file before they work on it and use the HTML Comment approach to the HTML::Template tags (instead of using the non-standard tagset). Our graphics department uses DreamWeaver (our IT team has come up with other more appropriate names, such as DreamMangler or NightmareWeaver) and it has a tendency to mangle the template files, "fixing" the HTML source of the file. Using the HTML Comment approach helps, but it is not a complete solution (it doesn't like HTML comments inside other tags). We haven't found a complete solution yet ... we just grin and bear it and after the graphics department is done, we trough through the source and undo the damage.

Even with HTML::Template, we were spending a lot of unnecessary time (in retrospect) hacking and evolving scripts to handle complex applications and that's where CGI::Application comes into play. I came across an article on perl.com written by Jesse Erbaum, the author of CGI::Application, an instigator to push us to evolve in that direction and what a blessing that has been for us.

Enough with the background - let's start by looking at a HelloWorld application in three variations. The helloworld-simple.cgi program is what we would write when we first started out (with CGI.pm). The helloworld.cgi, HelloWorld.pm and hello.TMPL files show two different approaches - one using HTML::Template and the other, not. In terms of lines of code, the helloworld-simple.cgi program looks like the way to go, but for some reason, enterprise applications demand a little more complexity than a "Hello World!" message and that's where CGI::Application really shines.

The steps we take whenever starting a new CGI::Application Web application are as follows: 1) Mock-up the screens in HTML that will be involved in the application and map out the flow between them, 2) Write an instance script, 3) Write the CGI::Application framework Perl module, 4) Code, Test, Debug & Repeat until done and 5) Deploy/Roll out your code (remove the debug aspects).

At this point in the presentation, I will be writing code on the fly. I have coded a few examples of what might transpire in the Code Appendix, but due to the dynamic nature of the presentation, the actual presentation code may differ. Please e-mail me for the presentation code, if you would like to review that as well, though the points I make in my presentation can be seen in the Appendix.

So let's write a new CGI::Application and we will follow those steps. In our example, we will code an Account Manager with two basic steps, one a login page and the other, a welcome page. So we first mock-up the two steps as html pages - more specifically, HTML::Template pages. The first page won't need any dynamic variables, so we could just make it a static HTML page and link it into the application through the form's ACTION tag. However, if you go ahead and make it a template page, then if your needs change for that first step (such as putting a time in there), then you won't have to worry about converting the page/step later. This is shown in login.TMPL and main.TMPL. I find it a good practice when writing HTML::Template files to just put <>'s whenever I'm going to show something dynamic and then go back later to fill in the <>'s with the TMPL_VAR tagset, just so they don't confuse me later.

Then the next step is to write an instance script that will be requested from the Web. This is a very simple script that uses your CGI::Application subclass and run()'s it. I usually initially sprinkle in debugging code (lines 3,4,7,8 in am.cgi) such that whenever I make an error, it will be easily seen. When I roll out the code, I will take those lines out. Also note that in line 5, I have to declare which library path to search in - if you install/save your CGI::Application .pm file in a non @INC path, you will need to do the same.

The next step is to go ahead and write a CGI::Application subclass framework. Think about where you want to save/install this file first ... I usually use a non-Web path (ie: /usr/local/lib) so that Web users cannot access the .pm file directly. Also, think about the organization of the file. Will this be a site-wide general module or will it apply to a sliver of your site? For example, I might have a general database module for CRM efforts named BrusselSprouts.pm (just kidding - seeing if you are paying attention ... really called CRM.pm) and I'll put that in /usr/local/lib/CRM.pm. I also might have a Classifieds module for our QSR Web site called Classifieds.pm and I'll put that in /usr/local/lib/QSR/Classifieds.pm. In both cases, whenever I want to use those modules, I need to: `use lib '/usr/local/lib';` and then per module, I need to use them in a naming convention that matches their path from that root. CRM.pm would be referenced by `use CRM;` Classifieds.pm would be referenced by `use QSR::Classifieds;`.

So now that you have an idea of where to put the file, let's write it! Your first line should declare its package name, matching how you're going to `use` it in the instance script (and possibly elsewhere). The next line should be `use base 'CGI::Application';`, which tells Perl that this class is a subclass of CGI::Application.

After this, there are two methods that you need to write - a `setup` method and a runmode method (name is based on your choice). The `setup` method is used to define the various runmodes that your application will support. You also need to define the `start_mode` and `mode_param` (using the respective methods) so that the underlying framework will do its job correctly. I usually have one runmode per screen in the mockup, though you can certainly use multiple templates per runmode and/or consolidate screens into one template/runmode. When you define the `run_modes`, there are various ways to do it. I use the latest method, by creating an array reference to a list of one-word names, which match both the parameter value and the runmode method name.

The way the CGI::Application framework works is that it is expecting a name/value pair in the CGI parameters, where the name matches the value you specify in the `mode_param` method ('rm' in my example) and the value will dictate which runmode method is called. If there is no name/value pair that matches in the CGI parameters, then it defaults to the `start_mode` that you defined in the `setup` method.

Defining a run mode method is pretty simple. You can expect a reference to the class as a parameter that's passed into the method via `@_` and you need to return the output that will be presented to the Web user. Note that you aren't to print to STDOUT, worry about HTTP headers (unless you're outputting something besides text/html) or anything like that. You can use the reference to the class that's passed in as you would a CGI instance, meaning that you can get to the Web user's data via the parameter. Now you don't have to use HTML::Template - you can put together your own HTML output in your method and return that. You can see an either/or example in the HelloWorld.pm file, between the `beginning` and `beginning_template` methods.

Once you have the basic framework done, give it a shot on your Web browser and see if you're all good. If so, you can proceed with developing the real functionality of your application, integrating with DBI or Text::CSV_XS for example.

As you learn and grow in your use of CGI::Application, you may want to check out other frameworks, such as CGI::Application::ValidateRM (which will validate the user's data per run mode) or look at mod_perl to speed things up. I have yet to successfully understand mod_perl (that's one of the reasons I'm here at ApacheCon, too - to learn mod_perl), but it's very easy to switch an application over to mod_perl if you've written it in the CGI::Application framework - you can simply use the instance script code in your Apache directive for the Location.

That about wraps it up for this introduction to CGI::Application - I hope you either enjoyed my presentation and/or learned something from this handout. Again, feel free to e-mail me with your questions and I hope I've given you the opportunity to pursue CGI::Application further and incorporate it into your Web projects.

Bibliography

Erlbaum, Jesse. “CGI::Application” cpan.org. 2 June 2003
<http://search.cpan.org/~jerlbaum/CGI-Application-3.1/Application.pm>

Erlbaum, Jesse. “Using CGI::Application” perl.com. 5 June 2001
<http://www.perl.com/pub/a/2001/06/05/cgi.html>.

Appendix: Code Examples

helloworld-simple.cgi

```
#!C:\Perl\bin\perl.exe -Tw

use strict;
use CGI;
use CGI::Carp qw ( fatalsToBrowser );

my $query = new CGI;
my $string = "Hello World! (from simple CGI)";

print $query->header;
print <<_EOF_;
   <html>
   <head>
   <title>Testing Application</title>
   </head>
   <body>

   This is the message from our simple CGI application:
   <i>$string</i>
   <p>
   </body>
   </html>
_EOF_
```

helloworld.cgi

```
#!C:\Perl\bin\perl.exe -wT

$|++;
use CGI::Carp qw( fatalsToBrowser );
use lib './';
use strict;
open( STDERR, ">&STDOUT" );
print "Content-type: text/html\n\n";

use HelloWorld;

my $hello = HelloWorld->new();
$hello->run();
```

HelloWorld.pm

- package HelloWorld
  - setup
  - beginning
  - beginning_template
- EOF

```
 1 package HelloWorld;
 2 use base 'CGI::Application';
 3 use strict;
 4
 5 sub setup {
 6    my $self = shift;
 7
 8    $self->run_modes( [ 'beginning', 'beginning_template' ] );
 9    $self->start_mode( 'beginning_template' );
10    $self->mode_param( 'rm' );
11 }
12
13 sub beginning {
14    my ( $self, $query, $string, $output );
15    $self = shift;
16    $query = $self->query;
17    $string = "Hello World (from beginning())!";
18
19    $output = <<_EOF_;
20    <html>
21    <head>
22    <title>Testing Application</title>
23    </head>
24    <body>
25
26    This is the message from our simple CGI application:
27    <i>$string</i>
28    <p>
29    </body>
30    </html>
31 _EOF_
32
33    $output;
34 }
35
36 sub beginning_template {
37    my ( $self, $query, $template, %params );
38    $self = shift;
39    $query = $self->query;
40
41    $template = $self->load_tmpl( 'hello.TMPL',
42                                  'die_on_bad_params' =>      0,
43                                  'associate'         =>      $query );
44
45    $params{ 'message' } = "Hello, World! (from template)\n";
46    $template->param( %params );
47    $template->output;
48 }
49
50 1;
```

hello.TMPL

```
 1 <html>
 2 <head>
 3 <title>Testing Application</title>
 4 </head>
 5 <body>
 6
 7 This is the message from our CGI::Application module: <i>
 8 <!-- TMPL_VAR NAME="message" --></i>
 9 <p>
10
11 </body>
12 </html>
```

am.cgi

```
 1 #!C:\Perl\bin\perl.exe -wT
 2
 3 $|++;
 4 use CGI::Carp qw( fatalsToBrowser );
 5 use lib './';
 6 use strict;
 7 open( STDERR, ">&STDOUT" );
 8 print "Content-type: text/html\n\n";
 9
10 use AccountManager;
11
12 my $app = AccountManager->new();
13 $app->run();
```

AccountManager.pm

- package AccountManager
  - setup
  - login
  - main
- EOF


```
 1 package AccountManager;
 2 use base 'CGI::Application';
 3 use strict;
 4
 5 sub setup {
```

```
 6    my $self = shift;
 7
 8    $self->run_modes( [ 'login', 'main' ] );
 9    $self->start_mode( 'login' );
10    $self->mode_param( 'rm' );
11 }
12
13 sub login {
14    my ( $self, $template );
15    $self = shift;
16
17    $template = $self->load_tmpl( 'login.TMPL' );
18    $template->output;
19 }
20
21 sub main {
22    my ( $self, $query, $template );
23    $self = shift;
24    $query = $self->query;
25
26    if ( $query->param( 'password' ) ne 'purdy' ) {
27        return login( $self );
28    }
29
30    $template = $self->load_tmpl(    'main.TMPL',
31                                     'associate' => $query
32        );
33    $template->output;
34 }
35
36 1;
```

login.TMPL

```
 1 <html>
 2 <head>
 3 <title>Account Manager :: Login</title>
 4 </head>
 5 <body>
 6 <p>
 7 Login below to access your account:
 8 </p>
 9
10 <form method="post">
11
12 <p>
13 Username: <input type="text" name="username"><br>
14 Password: <input type="password" name="password">
15 </p>
16
17 <p>
18 <input type="hidden" name="rm" value="main">
19 <input type="submit" value="Login">
20 </p>
21
22 </form>
23 </body>
24 </html>
```

main.TMPL

```
 1 <html>
 2 <head>
 3 <title>Account Manager :: Your Account</title>
 4 </head>
 5 <body>
 6
 7 <p>
 8 Welcome, <!-- TMPL_VAR NAME="username" -->!
 9 </p>
10
11 </body>
12 </html>
```

This presenation has ended.
Click anywhere to return.

_

